FINAL PROJECT REPORT OF PROGRAMMING FUNDAMENTALS

PROJECT TITLE:

"Slither Speller"

Submitted By:

Osailah Atif (25k-0789) and Fatima Javed (25k-0594)

Submitted To:

Miss Kinza Mushtaq

—

Submission Date:

21-Nov-2025

—

Department:

Computer Science
(Fall 2025)

FAST NUCES

ABSTRACT

Slither Speller is a small console game we built in C by mixing two simple ideas: the classic Snake movement and a basic word-making challenge. Instead of eating apples, our snake picks up random letters that appear on the grid. After collecting three of them, the player tries to make a real word and submits it. If the word exists in our small dictionary, the snake reacts happily; if not, it basically "roasts" the player with a disappointed message. The whole purpose of the project was to practice the basic concepts we learned in Programming Fundamentals, like arrays, strings, structures, loops, and simple game logic. By the end, we were able to create a game that is light, fun, and a bit silly, but it does show how even a very old game like Snake can be changed into something different.

1. INTRODUCTION

As for our first project, we wanted to make a simple game that is also a little different. Snake is easy to make in C, that's why we decided to mix it with a small word game, which we named Slither Speller. In this game, the snake collects letters instead of food. After picking three letters, the player tries to make a real word. If the word is right, the snake celebrates. If it is wrong, the snake reacts in a funny way. It is a small project, but it was a fun way to put what we learned into practice.

2. OBJECTIVES

Through this project, we aimed to:

- Make the snake collect letters, store them and let the player submit words using ENTER.
- Check words from a small dictionary of 40 words and give feedback with messages and ASCII art.
- Implement a strike system, incorrect submission of words increases strikes, and reaching 3 strikes ends the round.
- Make a requirement from the player to complete up to six words to win.
- Reinforce programming concepts like loops, arrays, functions, random number generation and keyboard input handling.

3. SYSTEM DESIGN

3.1 System overview:

Flow of the program:

- Start.
- Initialize the grid
- Place the snake at the center.
- Place random letters on the board for the snake to collect.
- End the round if the player forms 6 correct words or reaches 3 strikes.
- Show the win or lose message.
- Stop.

3.2 Algorithm:

- Start the game.
- Initialize board, snake, score, strikes and dictionary of 40 words.
- Place random letters on the board.
- Repeat until round ends, read player input:
  - Arrow keys to move the snake.
  - Enter key for the submission of the collected words.

Check for collisions:

- If the snake hits a wall or itself then end the game and display the lose message.

Check for letter collision:

- If the snake reaches a letter, add it to the collected letters and place a new letter on the board from the dictionary.

Check for correct word submission:

-If the collected letters form a correct word, increase score and correct word count. Show positive feedback.

-If the word is incorrect add a strike and show negative feedback.

- End the round if:
  -6 words are formed then player wins.
  -3 strikes, player loses.

3.3 Input and Output:

INPUT:

- Arrow keys for movement.
- Enter key to submit collected letters as a word.

OUTPUT:

- Snake and letters on the board.
- Score, correct words, and number of strikes.
- Feedback messages and ASCII art for correct or wrong words.
- Win or lose message.

4. IMPLEMENTATION

LANGUAGE: C
COMPILER/IDE: VS CODE with MinGW

4.1 Key Features:

•Snake moves in four directions and grows when it collects letters.
•Letters from the dictionary appear on the board for the player to collect.
•Collected letters can be submitted as a three-letter word.
•A dictionary of 40 words checks whether the submitted word is correct.
•Plus 10 points for each correct word.
•3 wrong words end the round.
•Fun ASCII art and messages show snake reactions to correct or wrong words.
•Rounds end when the player submits six correct words or reaches three strikes.

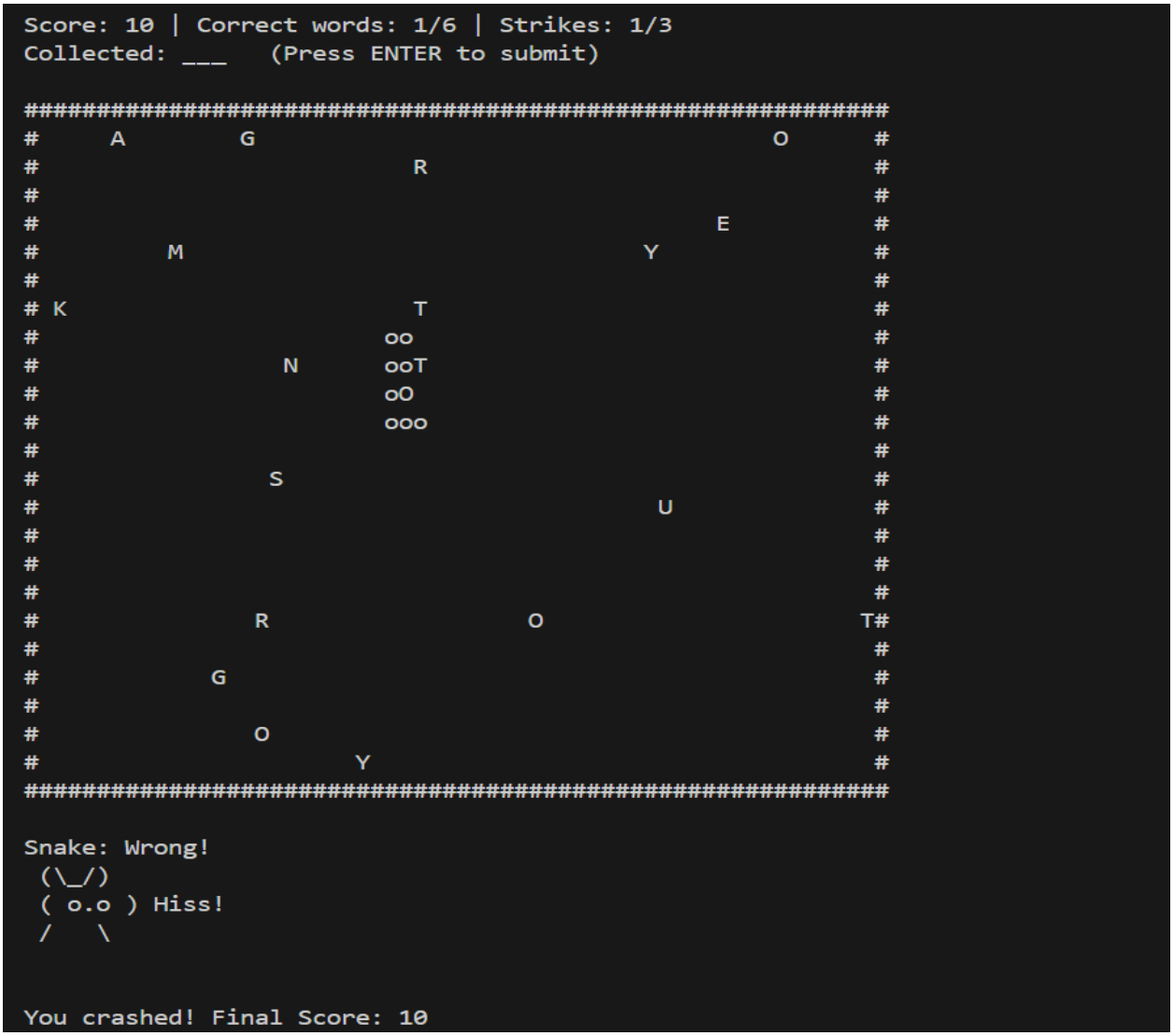
4.2 Code Snippet:

```
int main(){
    srand(time(NULL));

    initBoard();
    initSnake();

    system("cls");
    printf("Welcome to SlitherSpeller!\n");
    printf("Collect letters ? Press ENTER to submit a word.\n");
    printf("Find %d correct words to win.\n", WORDS_REQUIRED);
    printf("Make %d wrong submissions ? you lose.\n\n", MAX_STRIKES);
    printf("Press any key to start...");
    _getch();

    gameLoop();
    return 0;
}
```

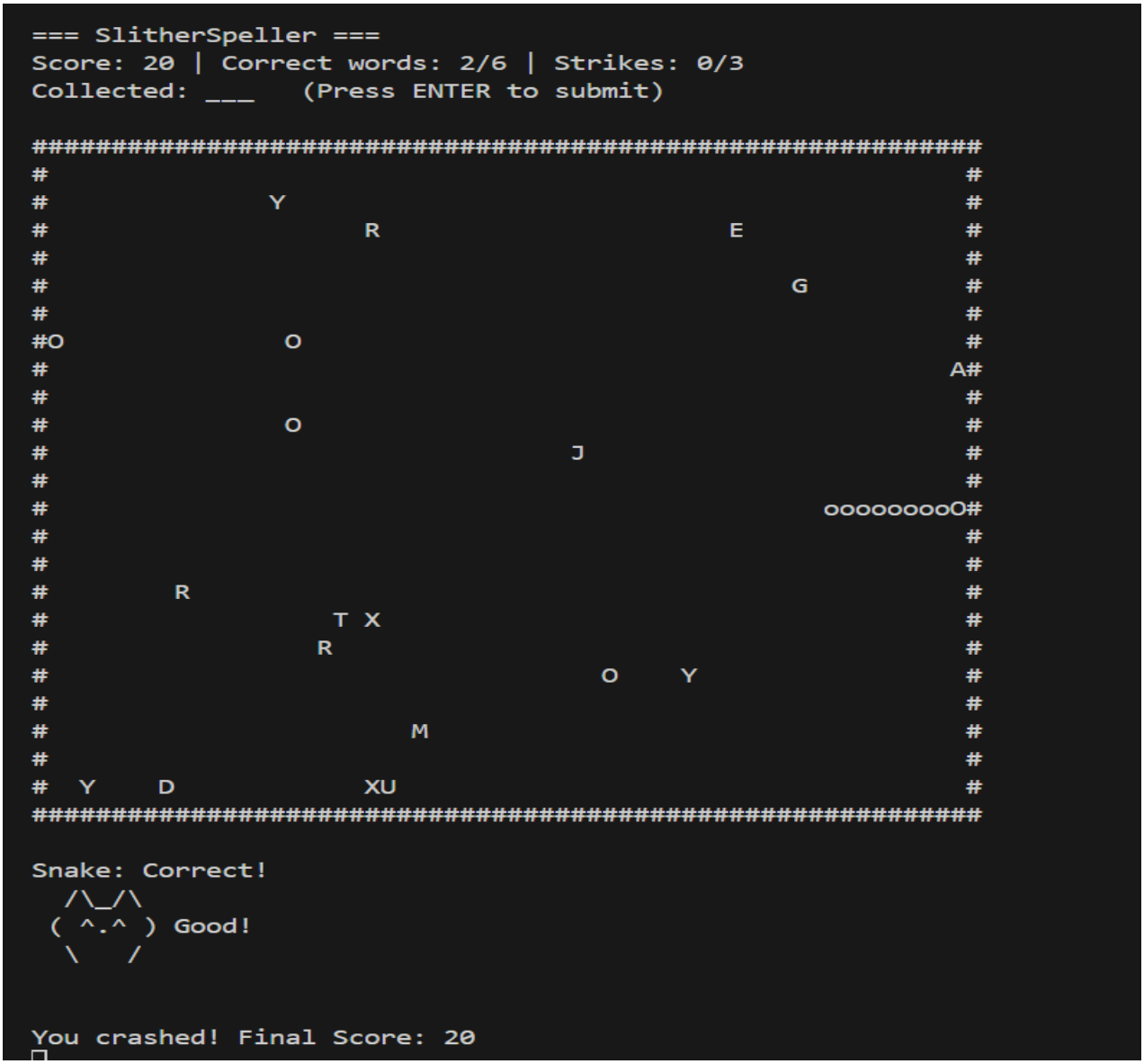
4.3Sample Output:

5. Testing and Results

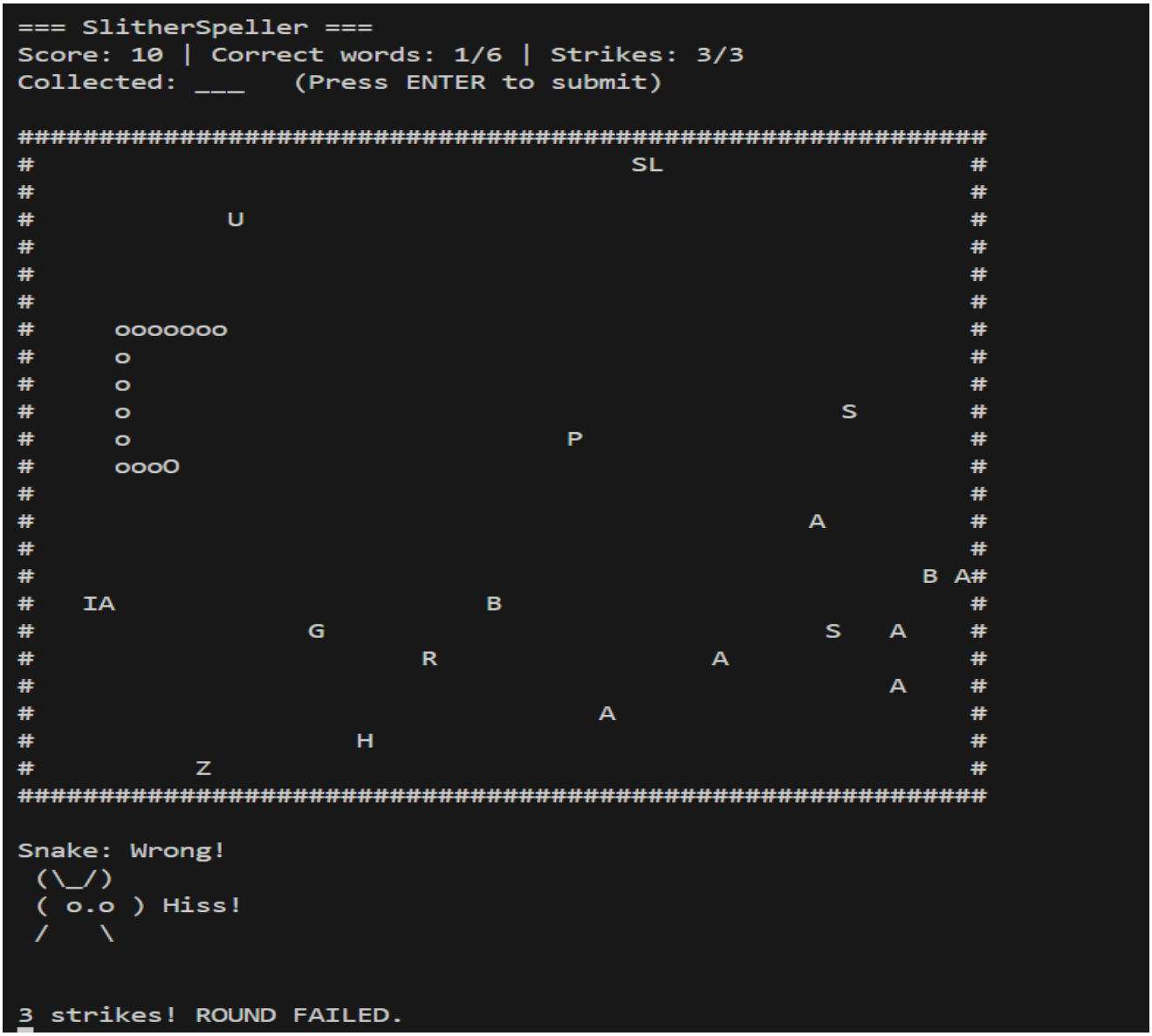
TEST NO: 01



TEST NO: 02


=== SlitherSpeller ===
Score: 20 | Correct words: 2/6 | Strikes: 0/3
Collected: ___ (Press ENTER to submit)
Snake: Correct!
(^.^) Good!
You crashed! Final Score: 20

TEST NO: 03


=== SlitherSpeller ===
Score: 10 | Correct words: 1/6 | Strikes: 3/3
Collected: ___ (Press ENTER to submit)
SL
U
oooooooo
o
o
o
o
oooO
S
P
A
B A
IA
B
G
S A
R
A
A
A
H
Z
Snake: Wrong!
(_/)
(o.o) Hiss!
/ \
3 strikes! ROUND FAILED.

OBSERVATION:

- The snake moves correctly in all directions.
- Letters are collected properly, and new ones appear randomly.
- Scores increase for correct words, and strikes are counted for wrong ones.
- Feedback messages and ASCII art make the game interactive.
- Rounds end correctly after 6 words or 3 strikes.
- Collisions with walls or the snake itself are detected accurately.

6. CONCLUSION, LIMITATIONS, AND REFERNCES

- **Conclusion**

Slither Speller successfully combines Snake with a word puzzle. The game works smoothly with proper snake movement, letter collection, scoring, strikes, and fun feedback. It shows how basic C programming concepts can create an interactive and enjoyable game.

- **Limitations**

1. This game only uses 3 letter words and a 40 word dictionary.
2. It runs on the console with no graphics.
3. The difficulty and speed are fixed.
4. No option to save or resume the game.

- **References**

https://www.w3schools.com/c/c_projects.php → w3schools
https://labex.io/tutorials/c-creating-a-snake-game-in-c-298831 → LabEx